CENG 218 – Labwork 5

Design, implement and test a Stack class in C++. In this lab work, we will help you in designing, implementing, and testing a Stack class in C++. It will also help you with understanding how to apply a stack object to an application.

Implementation

Push

The push operation inserts an element on the top of the stack. The recently added element is always at the top of the stack. Before every push, we must ensure whether there is a room for a new element.

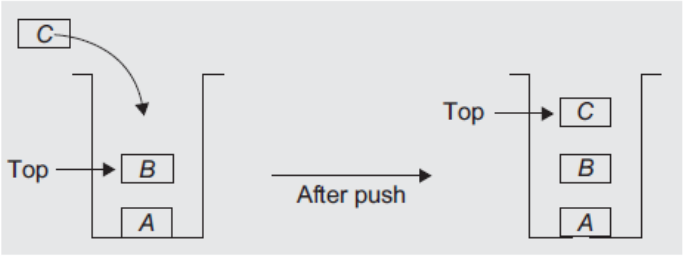

Fig. The push operation

When there is no space to accommodate the new element on the stack, the stack is said to be *full*. If the operation push is performed when the stack is full, it is said to be in *overflow* state, that is, no element can be added when the stack is full. The push operation modifies the *top* since the newly inserted element becomes the topmost element

Pop

The pop operation deletes an element from the top of the stack and returns the same to the user. It modifies the stack so that the next element becomes the top element.

When there is no element available on the stack, the stack is said to be *empty*.

If pop is performed when the stack is empty, then the stack is said to be in an *underflow state*

Peek (getTop/itemAtTop)

The getTop operation gives information about the topmost element and returns the element on the top of the stack.

In this operation, only a copy of the element, which is at the top of the stack, is returned. Hence, the top is still set to the same element

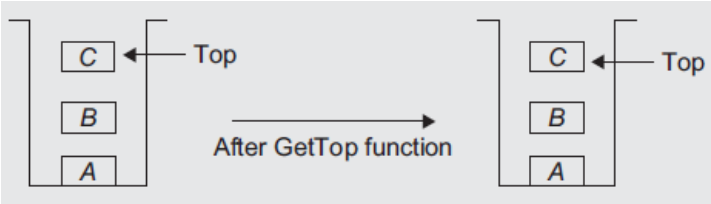

Fig. The getTop operation

This is the key difference between the pop and getTop operations. The getTop operation does not modify the variable *top*. It signals the stack underflow error if the stack is empty.

Stack ADT Specifications:

- Create Stack—Initializes the stack to an empty state. (Constructor)
- **isEmpty**—Determines whether the stack is empty. If the stack is empty, it returns the value true; otherwise, it returns the value false.
- **isFull**—Determines whether the stack is full. If the stack is full, it returns the value true; otherwise, it returns the value false.
- **push**—Adds a new element to the top of the stack. The input to this operation consists of the stack and the new element. Prior to this operation, the stack must exist and must not be full.
- **top**—Returns the top element of the stack. Prior to this operation, the stack must exist and must not be empty. (Sometimes this method is named as peek or itemAtTop)
- **pop**—Removes the top element of the stack. Prior to this operation, the stack must exist and must not be empty.

Array Based Implementaion

A stack can be implemented using both a static data structure (array) and a dynamic data structure (linked list). The simplest way to represent a stack is by using a one-dimensional array. A stack implemented using an array is also called a *contiguous stack*.

Stacks and their applications

Binary To Decimal Conversion

(Converting a Number from Binary to Decimal) The language of a computer, called machine language, is a sequence of 0s and 1s. When you press the key A on the keyboard, 01000001 is stored in the computer. Also, the collating sequence of 'A' in the ASCII character set is 65. In fact, the binary representation of A is 01000001 and the decimal representation of A is 65.

The numbering system we use is called the decimal system, or base 10 system. The numbering system that the computer uses is called the binary system, or base 2 system. The purpose of this exercise is to write a function to convert a number from base 2 to base 10.
To convert a number from base 2 to base 10, we first find the weight of each bit in the binary number. The weight of each bit in the binary number is assigned from right to left. The weight of the rightmost bit is 0. The weight of the bit immediately to the left of the rightmost bit is 1, the weight of the bit immediately to the left of it is 2, and so on. Consider the binary number
1001101. The weight of each bit is as follows:

Weight 6 5 4 3 2 1 0
1 0 0 1 1 0 1

We use the weight of each bit to find the equivalent decimal number. For each bit, we multiply the bit by 2 to the power of its weight, and then we add all of the numbers. For the binary number 1001101, the equivalent decimal number is

$$= 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$
$$= 64 + 0 + 0 + 8 + 4 + 0 + 1$$
$$= 77$$

To write a program that converts a binary number into the equivalent decimal number, we note two things: (1) The weight of each bit in the binary number must be known, and (2) the weight is assigned from right to left. Because we do not know in advance how many bits are in the binary number, we must process the bits from right to left. After processing a bit, we can add 1 to its weight, giving the weight of the bit immediately to its left. Also, each bit must be extracted from the binary number and multiplied by 2 to the power of its weight. To extract a bit, you can use the mod operator.

```
int convertBinaryToInt(char binString[])
{
        Stack<int> s;
        int res;
        //your implementation


        return res;
```

Postfix Calculator:

Write a program that will read postfix expression that may contain only integer numbers and operators (consider only binary operators: /, *, -, and +) which are delimited by single or more white space. Then the program will calculate the value of the expression and print it.
(If input postfix expression is not valid appropriate error message will be printed such as missing operand, missing operator or invalid symbol in the expression)

Infix to Postfix
Write a program that converts an infix expression into an equivalent postfix expression. The rules to convert an infix expression into an equivalent postfix expression are as follows:
Suppose infx represents the infix expression and pfx represents the postfix expression.

The rules to convert infx into pfx are as follows:

1) Initialize pfx to an empty expression and also initialize the stack.
2) While there are symbols in the infx expression, Get the next symbol, sym, from infx.
  - **a)** If sym is an operand, append sym to pfx.
  - **b)** If sym is (, push sym into the stack.
  - **c)** If sym is), pop and append all the symbols from the stack until the most recent left parenthesis. Pop and discard the left parenthesis.
  - **d)** If sym is an operator:
    - i) If s is not empty ,Pop and append all the operators from the stack to pfx that are above the most recent left parenthesis and have precedence greater than or equal to sym.
    - ii) Push sym onto the stack.
3) After processing infx, some operators might be left in the stack. Pop and append to pfx everything from the stack.

In this program, you will consider the (binary) arithmetic operators: +, -, *, and /, operands are single letters. In valid infix expression operands and operators are delimited with one or more white space. You may assume that the expressions you will process are error free.
Design a class that stores the infix and postfix strings. The class must include the following operations:
-infix: string
-postfix: string
+getInfix—Stores the infix expression
+showInfix—Outputs the infix expression
+showPostfix—Outputs the postfix expression
Some other operations that you might need are the following:
+convertToPostfix—Converts the infix expression into a postfix expression. The resulting postfix expression is stored in pfx.
-precedence—Determines the precedence between two operators. If the first operator is higher or equal precedence than the second operator, it returns the value true; otherwise, it returns the value false.
Include the constructors and destructors for automatic initialization and dynamic memory deallocation.

Test your program on the following five expressions:
A + B - C;
(A + B) * C;

(A + B) * (C - D);
A + ((B + C) * (E - F) - G) / (H - I);
A + B * (C + D) - E / F * G + H;
For each expression, your answer must be in the following form:
Infix Expression: A + B - C;
Postfix Expression: A B + C -